



ساختمان داده‌ها و الگوریتم‌ها

کویز اول - پیچیدگی زمانی و بازگشتی

شماره دانشجویی:

نام و نام خانوادگی:

تاریخ: ۱۶/۱۲/۱۴۰۱

۳۳ نمره

۱. پیچیدگی زمانی

پیچیدگی زمانی کدهای زیر را بررسی کنید.

```
a)
int func1(int n, int m) {
    int ans = 0;
    for(int i = 1; i <= n; ++i) {
        int var = n;
        while(var > 0) {
            // do some O(1) operation.
            var /= 2;
        }
    }

    for(int j = 1; j <= m; ++j)
        ans++;
    return ans;
}

b)
void func2(int n){
    for (int i = 1; i <= n; i++) {
        int j = 1;
        while (j*j <= i) {
            j++;
        }
    }
}
```

پاسخ:

(a) در این کد، یک حلقه‌ی درونی با مرتبه‌ی زمانی $O(\log N)$ داخل حلقه‌ی اول وجود دارد. بنابراین پیچیدگی کلی حلقه‌ی اول برابر است با $O(N \log N)$.

حلقه‌ی دوم به‌تنهایی دارای مرتبه‌ی $O(M)$ می‌باشد. از آنجایی که N و M دو متغیر مستقل از هم هستند و نمی‌توان یکی را نسبت به دیگری کوچک یا بزرگ فرض کرد. پیچیدگی زمانی کل برنامه برابر است با:

$$O(N \log N + M)$$

(b) در این کد، حلقه‌ی بیرونی i از ۱ تا n اجرا می‌شود، بنابراین تعداد تکرارهای آن برابر n است.

حلقه‌ی درونی، مقدار j را از ۱ شروع می‌کند و تا زمانی که $j^2 \leq i$ ادامه می‌یابد. در واقع، حلقه‌ی درونی تقریباً $\sqrt{i}$ بار اجرا می‌شود. بنابراین مجموع کار انجام شده برای کل حلقه‌ها برابر است با:

$$T(n) = \sum_{i=1}^n O(\sqrt{i}) = O(n\sqrt{n})$$

۲. پیچیدگی زمانی

۳۳ نمره

پیچیدگی زمانی کدهای زیر را بررسی کنید.

a)

```
void Code1(int n, int m) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            int k = 1;
            while (k < n * m) {
                k *= 2;
            }
        }
    }
}
```

b)

```
void Code2(int n) {
    double s = 2.0;
    while (s < n) {
        s = s * s;
    }
}
```

پاسخ:

(a) در این کد، حلقه‌ی بیرونی n بار اجرا می‌شود و حلقه‌ی درونی نیز m بار اجرا می‌شود.

درون آن‌ها، یک حلقه‌ی while وجود دارد که در هر تکرار مقدار k را دو برابر می‌کند تا زمانی که $k \geq n \times m$ شود. بنابراین این حلقه دارای مرتبه‌ی زمانی $O(\log(n \times m))$ است. در نتیجه، پیچیدگی کلی برنامه برابر است با:

$$O(nm \log(nm))$$

(b) در این کد، متغیر s با مقدار اولیه ۲ شروع می‌شود و در هر تکرار با خودش ضرب می‌شود: $s = s \times s$.

اگر تعداد تکرارها را k در نظر بگیریم، داریم:

$$s = 2^{2^k}$$

حلقه تا زمانی ادامه می‌یابد که $s \geq n$ ، بنابراین:

$$2^{2^k} \geq n \implies 2^k \geq \log_2 n \implies k \geq \log_2(\log_2 n)$$

بنابراین پیچیدگی زمانی این حلقه برابر است با:

$$O(\log \log n)$$

۳۴ نمره

۳. پیچیدگی زمانی

پیچیدگی زمانی تابع زیر را بررسی کنید.

$$T(n) = \sqrt{n}T(\sqrt{n}) + n$$

پاسخ:

فرض می‌کنیم که $n = 2^k$ است آنگاه نتیجه می‌شود که $\sqrt{n} = 2^{k/2}$ است و همچنین میتوان گفت که $k = \log n$ است. حالا با کمک تغییر متغیر هایی که دادیم به سراغ حل مسئله میرویم:

$$T(2^k) = 2^{k/2}T(2^{k/2}) + 2^k$$

حالا دو طرف معادله را بر 2^k تقسیم می‌کنیم.

$$\frac{T(2^k)}{2^k} = \frac{2^{k/2}T(2^{k/2})}{2^k} + 1$$

با ساده سازی به معادله زیر میرسیم:

$$\frac{T(2^k)}{2^k} = \frac{T(2^{k/2})}{2^{k/2}} + 1$$

حالا تغییر متغیر $y(k) = \frac{T(2^k)}{2^k}$ را اعمال می‌کنیم و مسئله به مسئله زیر تبدیل میگردد:

$$y(k) = y\left(\frac{k}{2}\right) + 1$$

حالا این سوال با قضیه اصلی به سادگی قابل حل خواهد بود. جواب نهایی مسئله به صورت زیر است:

$$T(n) = \Theta(n \log \log n)$$